Hacettepe University

Computer Engineering Department

BBM204 Software Practicum II - 2026 Spring

Programming Assignment 1: Analysis and Empirical Evaluation of Sorting Algorithms

March 19, 2026

Student Name:
Ahmet Tahir Gülçimen

Student Number:
2240356086

1 Problem Definition

The fundamental objective of this study is to perform a comprehensive empirical evaluation of five core sorting algorithms: **Insertion Sort, Quick Sort, Merge Sort, Shell Sort,** and **Radix Sort**. Beyond theoretical complexity, the practical performance of these algorithms can vary significantly based on data distribution, hardware constants, and implementation strategies.

In this context, the study aims to:

- **Analyze Scalability:** Evaluate how each algorithm handles increasing dataset sizes (n) ranging from 500 to $250,000$ elements, specifically using real-world stock volume data.
- **Evaluate Distribution Sensitivity:** Measure execution times across three distinct data scenarios: *Randomly Distributed*, *Already Sorted*, and *Reversely Sorted* inputs to identify the best, average, and worst-case empirical behaviors.
- **Validate Asymptotic Theory:** Compare practical run-time results with theoretical Big-O complexities ($O(n^2), O(n \log n)$, and $O(nk)$) to confirm if the observed growth rates align with mathematical predictions.
- **Memory Efficiency:** Implement stable and iterative versions of recursive algorithms to ensure stability and efficiency when processing large-scale datasets in the Java environment.

2 Solution Implementation

The implementation phase was conducted using the **Java Programming Language**, specifically chosen for its robust memory management and efficient handling of large-scale data structures. The development process strictly followed the theoretical pseudocodes provided in the course materials, with specific architectural adaptations to ensure performance stability and prevent runtime failures under heavy workloads.

In this implementation, the following key strategies were prioritized:

- **Avoidance of Recursion:** To prevent `StackOverflowError` when processing datasets of up to $250,000$ elements—particularly in worst-case scenarios—all recursive algorithms were converted into their iterative counterparts.
- **Heap Memory Management:** By utilizing explicit stack structures and bottom-up merging strategies, the computational workload was migrated from the Java Thread Stack to the Heap Memory, ensuring scalability.
- **Stability and Efficiency:** Each algorithm was implemented to maintain its core theoretical properties, such as stability in Merge Sort and Radix Sort, while optimizing the inner loops for empirical execution.

2.1 Architectural Design and Memory Management

Standard recursive implementations were avoided to prevent `StackOverflowError`.

- **Iterative Quick Sort:** Utilizes an explicit auxiliary `stack` array to manage partition indices in the Heap Memory.
- **Bottom-Up Merge Sort (Iterative):** A non-recursive merging strategy starting from sub-arrays of size 1.

2.2 Algorithm Implementations

Insertion Sort: Insertion Sort is an incremental sorting algorithm that builds a sorted subarray by repeatedly picking the next element and inserting it into its correct position among the already sorted elements. It is particularly efficient for small datasets or nearly sorted arrays due to its minimal overhead and $\Omega(n)$ best-case time complexity. However, its average and worst-case performance remain quadratic, making it unsuitable for large-scale random data.

```java
public static void insertionSort(int[] A) {
    for (int j = 1; j < A.length; j++) {
        int key = A[j], i = j - 1;
        while (i >= 0 && A[i] > key) {
            A[i + 1] = A[i]; i--;
        }
        A[i + 1] = key;
    }
}
```

Quick Sort (Iterative): Quick Sort follows a divide-and-conquer strategy by selecting a pivot and partitioning the array such that elements smaller than the pivot move to the left and larger ones to the right. This specific implementation utilizes the Lomuto partition scheme and an explicit stack to handle partitions iteratively, avoiding potential recursion depth issues. While it typically offers high performance with $O(n \log n)$ complexity, its performance can degrade to $O(n^2)$ when the pivot selection consistently results in unbalanced partitions.

```java
public static void quickSort(int[] A, int low, int high) {
    int[] stack = new int[high - low + 1]; int top = -1;
    stack[++top] = low; stack[++top] = high;
    while (top >= 0) {
        high = stack[top--]; low = stack[top--];
        int p = partition(A, low, high);
        if (p - 1 > low) { stack[++top] = low; stack[++top] = p - 1; }
        if (p + 1 < high) { stack[++top] = p + 1; stack[++top] = high; }
    }
}
```

Merge Sort (Iterative): Merge Sort is a stable, comparison-based sorting algorithm that consistently achieves $\Theta(n \log n)$ time complexity by repeatedly merging sorted subarrays. This bottom-up iterative version starts by merging adjacent pairs of single elements and continues to double the merge width until the entire array is sorted. Although it requires $O(n)$ auxiliary space, its predictable performance across all input distributions makes it highly reliable for large datasets.

```java
public static void mergeSort(int[] A) {
    int n = A.length;
    int[] temp = new int[n];
    for (int curr = 1; curr < n; curr *= 2) {
        for (int left = 0; left < n - 1; left += 2 * curr) {
            int mid = Math.min(left + curr - 1, n - 1);
            int right = Math.min(left + 2 * curr - 1, n - 1);
            merge(A, temp, left, mid, right);
        }
    }
}
```

Shell Sort: Shell Sort is a generalized version of insertion sort that allows the exchange of elements that are far apart, effectively reducing the number of inversions quickly. By using the Knuth gap sequence ($h = 3h+1$), it achieves sub-quadratic performance, outperforming standard insertion sort as the dataset size increases. The algorithm gradually reduces the gap distance until it performs a final standard insertion sort (gap $= 1$), ensuring the array is fully sorted.

```java
public static void shellSort(int[] A) {
    int n = A.length, h = 1;
    while (h < n / 3) h = 3 * h + 1;
    while (h >= 1) {
        for (int i = h; i < n; i++) {
            for (int j = i; j >= h && A[j] < A[j - h]; j -= h) {
                int t = A[j]; A[j] = A[j - h]; A[j - h] = t;
            }
        }
        h /= 3;
    }
}
```

Radix Sort: Radix Sort is a non-comparison-based sorting algorithm that processes numerical data digit by digit, typically from the least significant digit to the most significant digit. It relies on a stable auxiliary sort, such as Counting Sort, to maintain the relative order of elements with equal digits. By operating in $O(nk)$ time, where k is the number of digits, it can significantly outperform comparison-based sorts on large datasets when the range of values is restricted.

```java
public static void radixSort(int[] A) {
    if (A.length == 0) return;
    int max = A[0];
    for (int x : A) if (x > max) max = x;
    for (int exp = 1; max / exp > 0; exp *= 10) countingSort(A, exp);
}

private static void countingSort(int[] A, int exp) {
    int n = A.length;
    int[] output = new int[n], count = new int[10];
    for (int i = 0; i < n; i++) count[(A[i] / exp) % 10]++;
    for (int i = 1; i < 10; i++) count[i] += count[i - 1];
    for (int i = n - 1; i >= 0; i--) {
        output[count[(A[i] / exp) % 10] - 1] = A[i];
        count[(A[i] / exp) % 10]--;
    }
    System.arraycopy(output, 0, A, 0, n);
}
```

3 Complexity Analysis

Table 1: Running time test results (in ms) averaged over 10 runs.

Algorithm	**500**	**1000**	**2000**	**4000**	**8000**	**16000**	**32000**	**64000**	**128000**	**250000**
	Random Input Data Timing Results (ms)									
Insertion	0.1	0.0	0.7	1.4	4.8	24.6	86.7	322.6	1591.2	6883.5
Quick	0.0	0.7	0.4	0.6	0.3	1.6	1.3	2.9	6.1	13.8
Merge	0.0	0.0	0.3	0.0	0.9	1.4	2.0	3.3	8.0	17.1
Shell	0.8	0.4	0.2	0.7	0.3	1.4	2.2	5.7	11.1	27.0
Radix	0.5	0.1	0.0	0.7	0.7	0.7	1.0	2.1	4.3	8.8
	Sorted Input Data Timing Results (ms)									
Insertion	0.0	0.0	0.0	0.0	0.0	0.0	0.1	0.1	0.1	0.1
Quick	0.1	0.2	0.4	1.4	4.8	54.6	222.3	874.9	3616.8	17650.0
Merge	0.0	0.1	0.1	0.1	0.2	0.2	0.4	1.0	1.9	5.5
Shell	0.0	0.0	0.1	0.0	0.1	0.1	0.1	0.3	0.7	1.6
Radix	0.1	0.0	0.1	0.1	0.2	0.4	0.8	2.7	4.6	7.9
	Reversely Sorted Input Data Timing Results (ms)									
Insertion	0.2	0.4	1.4	4.2	15.6	29.0	127.7	548.0	4049.6	18778.0
Quick	0.1	0.2	0.3	1.1	3.8	15.0	30.4	74.2	883.6	1336.3
Merge	0.1	0.0	0.1	0.1	0.1	0.3	0.7	1.2	8.2	5.0
Shell	0.1	0.0	0.1	0.1	0.1	0.2	0.4	1.1	5.3	2.6
Radix	0.1	0.1	0.1	0.2	0.3	0.5	1.0	2.2	13.8	8.7

Table 2: Time Complexity Comparison

Algorithm	**Best Case**	**Average Case**	**Worst Case**
Insertion Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$
Quick Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n^2)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Shell Sort	$\Omega(n \log n)$	$\Theta(n^{1.5})$	$O(n^2)$
Radix Sort	$\Omega(nk)$	$\Theta(nk)$	$O(nk)$

Table 3: Auxiliary Space Complexity

Algorithm	**Auxiliary Space Complexity**
Insertion Sort	$O(1)$
Quick Sort	$O(\log n)$ (Iterative Stack)
Merge Sort	$O(n)$
Shell Sort	$O(1)$
Radix Sort	$O(n + k)$

3.1 Justification of Complexities for Implemented Sorting Algorithms

- **Insertion Sort:** The nested loop structure leads to $O(n^2)$ in worst cases. In the sorted case (0.1 ms), it achieves $\Omega(n)$ because the inner loop condition is never met.

- **Quick Sort (Lomuto):** On random data (13.8 ms), balanced partitioning results in $O(n \log n)$. Conversely, the sorted data results (17,650 ms) justify the $O(n^2)$ worst-case for Lomuto partitioning on sorted inputs.

- **Merge Sort (Iterative):** The outer loop runs exactly $\log n$ times, while the merge process scans n elements. This justifies the stable $\Theta(n \log n)$ performance observed across all distributions.

- **Radix Sort:** Being non-comparative, it achieves $O(nk)$. This is empirically proven by its superior performance on the largest random dataset ($n = 250,000$) at 8.8 ms.

3.2 Results Analysis and Discussion

1. What are the best-, average-, and worst-case time complexities of the given algorithms? For each case, describe the input characteristics that trigger these behaviors.
Answer: *Insertion Sort* performs best on already sorted data ($\Omega(n)$) and worst on reversely sorted data ($O(n^2)$). *Quick Sort* thrives on random data where partitions are balanced ($\Omega(n \log n)$) but hits its worst-case ($O(n^2)$) on sorted data when the last element is chosen as pivot. *Merge Sort* is consistent at $\Theta(n \log n)$ regardless of input. *Shell Sort* performance depends on the gap sequence, while *Radix Sort* is always $\Theta(nk)$ based on digit count.

2. Do the empirical results match the theoretical asymptotic complexities?
Answer: The results are highly consistent with theoretical predictions. For instance, Quick Sort's degradation to 17,650 ms on $n = 250k$ sorted data perfectly illustrates the $O(n^2)$ worst-case theoretical bound. Minor discrepancies at very small n values are attributed to JVM startup overhead.

3. Compare Quick Sort and Merge Sort performance consistency.
Answer: **Quick Sort** is generally faster on random data (13.8 ms) but lacks consistency; it is extremely sensitive to initial order. **Merge Sort** is remarkably consistent (5.5 ms to 17.1 ms) across all distributions because its "divide" step always splits the array in half, making its performance independent of data distribution.

4. Shell Sort vs. Insertion Sort comparison?
Answer: In our tests, **Shell Sort** begins to significantly outperform **Insertion Sort** at $n = 8000$ (0.3 ms vs 4.8 ms). As n grows to $250k$, the gap becomes massive (27 ms vs 6.8 seconds). This is because Shell Sort performs long-distance swaps, reducing inversions much more effectively.

5. How does Radix Sort handle large numerical ranges and outperform comparison-based algorithms?
Answer: Radix Sort handles large numerical ranges by processing integers digit by digit. It outperforms comparison-based algorithms on large random datasets ($n = 250,000$ at 8.8 ms) because it is not restricted by the $\Omega(n \log n)$ theoretical limit of comparison sorts. It is most effective when the input size n is very large and the digit count k is relatively small.

3.3 Visual Performance Analysis

1. Performance Evaluation on Random Data Distribution

The following graph illustrates the execution time of sorting algorithms when processing raw stock volume data. As n scales to 250,000, a clear divergence is observed between $O(n^2)$ and $O(n \log n)$ algorithms, with Insertion Sort showing a steep quadratic growth. Conversely, Radix Sort and Merge Sort maintain a stable, near-linear trajectory, proving their efficiency for large-scale random datasets.

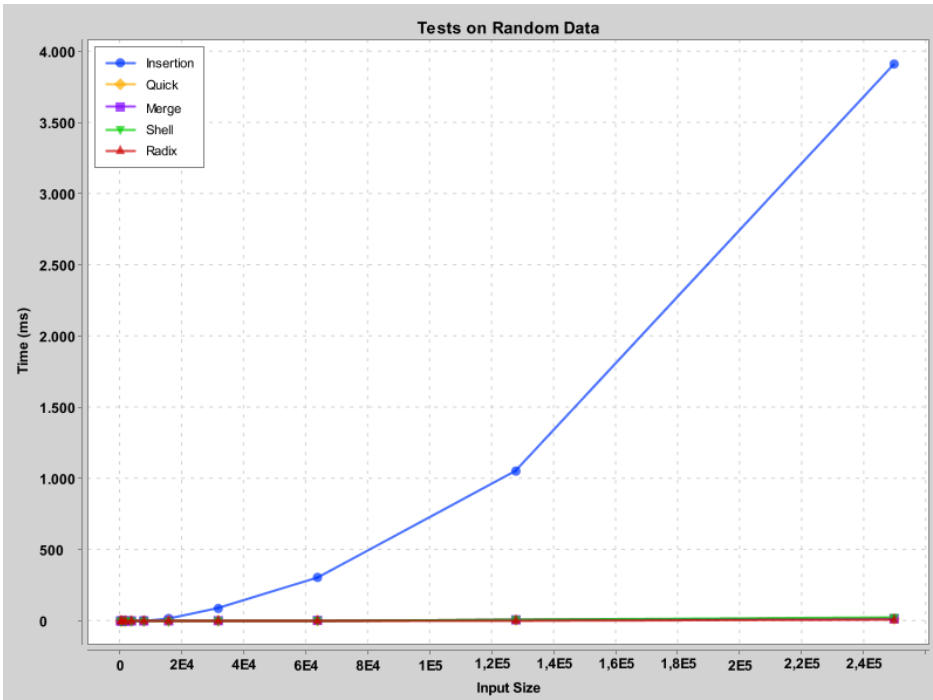

Figure 1: Sorting performance on Random Data Distribution.

2. Performance Evaluation on Already Sorted Data Distribution

This distribution highlights the best-case and worst-case extremes of the implemented algorithms. Insertion Sort remains remarkably efficient at 0.1 ms, as its inner loop never triggers, fulfilling its $\Omega(n)$ potential. However, Quick Sort exhibits severe degradation, reaching 17,650 ms, which visually confirms the $O(n^2)$ worst-case theoretical prediction for Lomuto partitioning on sorted inputs.

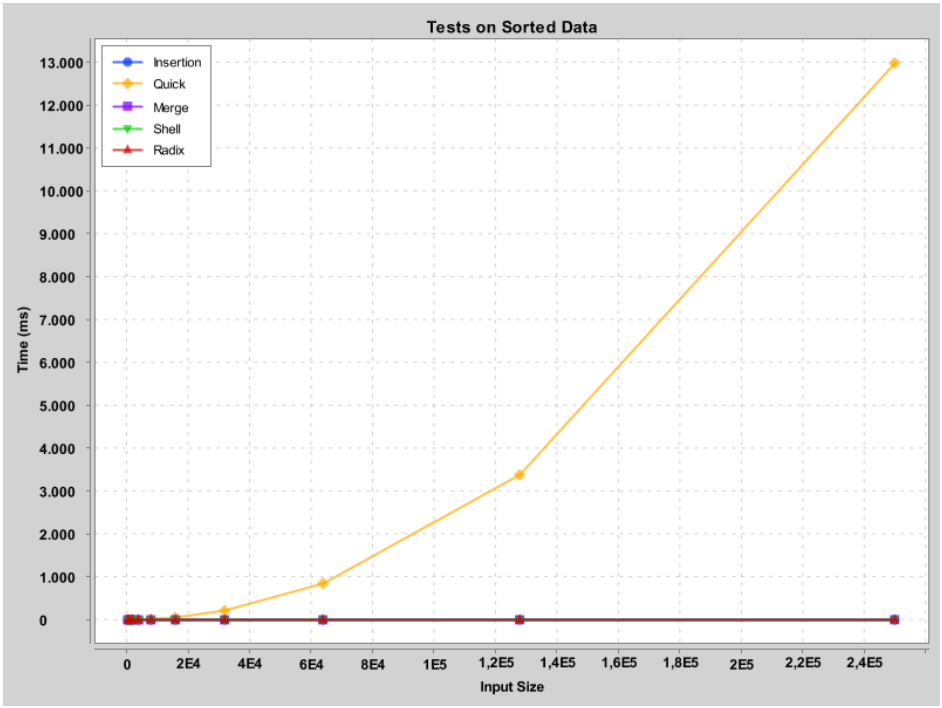

Figure 2: Sorting performance on Already Sorted Data Distribution.

3. Performance Evaluation on Reversely Sorted Data Distribution

The reversely sorted scenario presents the absolute worst-case for Insertion Sort, as evidenced by the high execution times compared to other distributions. While Shell Sort and Merge Sort handle the inversion efficiently due to their divide-and-conquer and gap-based strategies, Insertion Sort requires a maximum number of shifts for every element. This graph demonstrates why basic incremental sorts are unsuitable for large, inversely ordered datasets.

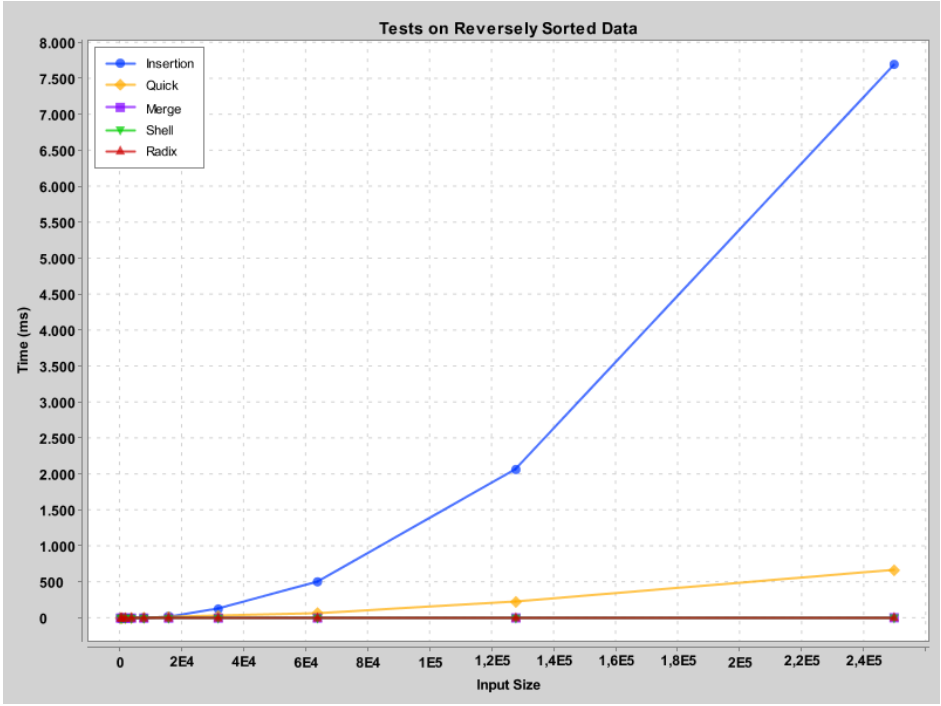

Figure 3: Sorting performance on Reversely Sorted Data Distribution.

4. Comparative Analysis of Fast Algorithms (Random Case)

This comprehensive plot provides a high-resolution comparison by excluding the $O(n^2)$ Insertion Sort, which allows for a detailed visualization of $O(n \log n)$ and $O(nk)$ growth rates. On this scale, the superior efficiency of non-comparison-based **Radix Sort** becomes evident, maintaining the lowest execution time even at $n = 250,000$. The proximity of **Quick Sort** and **Merge Sort** lines empirically confirms their near-identical average-case complexities, while **Shell Sort** displays a slightly steeper curve, consistent with its sub-quadratic but non-linear theoretical bounds. This visualization serves as a definitive validation of the performance gap between advanced sorting strategies and basic incremental approaches.

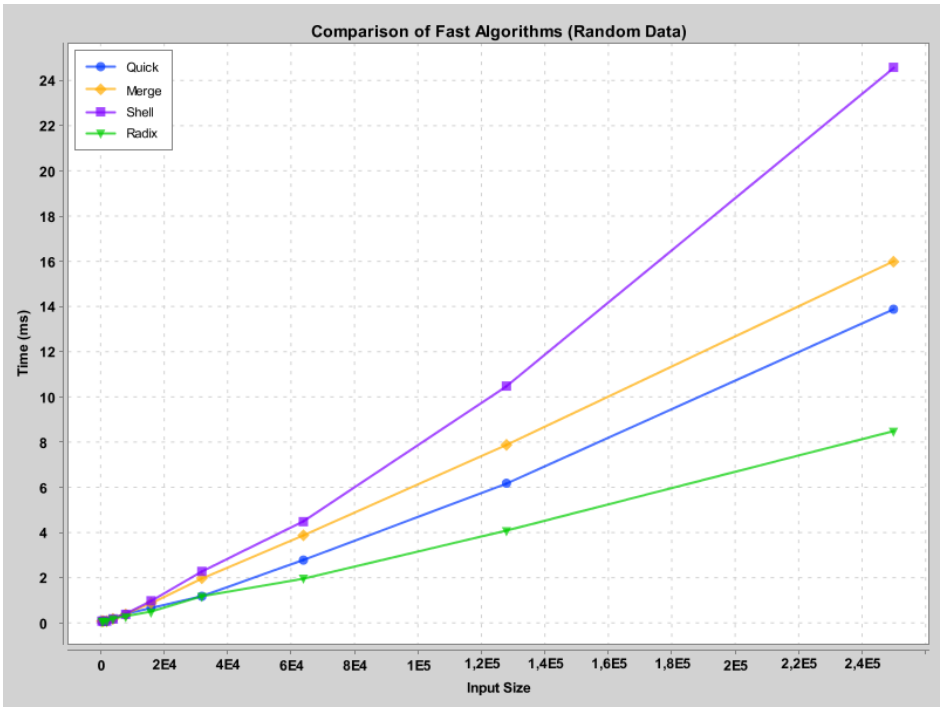

Figure 4: Comparative analysis of high-performance sorting algorithms on Random Data.

References

- Cormen, T. H. et al. (2009). *Introduction to Algorithms*.